CPE 314 — Software Design & Software Design Laboratory

Week 8 — Project Proposal

Due end of Week 8 session | 10% of the Finals Term Grade | Scored out of 100

Note: this proposal is now graded — 10% of the Finals Term Grade, scored out of 100 — see Week08_Checklist.docx for the full Student Self-Check and Instructor Grading breakdown.

Group Name / No.:	2
Members:	Justine Cuizon, Clarence Philip Elegino, Joshua Malaya, Gebrielle P. Mirasol, John Daevin S. Simbajon
Repository URL:	https://github.com/daevinsimbajon-web/cpe314-69167-capstone-team-02.git
Chosen Theme (circle one):	A B C D E F G H I J K L M N O P Q R (S) T ☐ Custom (attach approval)
API You Will Use:	Open Trivia Database — https://opentdb.com/api_config.php — no key required; example: GET opentdb.com/api.php?amount=10&category=9&difficulty=easy&type=multiple

See Project_Specification_and_Rubric.docx, Section 4, for what each letter (A-T) means and Section 4.1 for the matching verified public API and its auth requirement.

Reminder: no two groups in this class may build the same theme — confirm your choice isn't already taken before submitting.

1. One-Paragraph Pitch

(3-5 sentences: what does your app do, and who is it for?)

Trivia & Quiz Game Night Manager is a Java desktop app that helps friends, classmates, and community groups run organized trivia nights. Players enter their names, pick a category, and answer questions fetched live from the Open Trivia Database, with each question served in order and an “undo last answer” option for mistakes. The app saves every player, game session, and score in a local SQLite database, and it can import past score history from CSV files without freezing the interface. It then analyzes the data to show average score per category, most-missed questions, and score trends over time as labeled bar and line charts. It is built for casual quiz hosts and groups who want to track results across game nights instead of keeping score on paper.

2. Entity-Relationship Sketch

(List your entities, their key attributes, and how they relate — text notation is fine, e.g., Member(id, name) --1:M--> Loan(id, memberId FK, bookId FK) <--M:1-- Book(id, title))

Category --1:M--> Category (parent_id; a category has many subcategories)
Category --1:M--> Question (Question.category_id)
Category --1:M--> Score (Score.category_id)
Player ----1:M--> Score (Score.player_id)
GameSession -1:M--> Score (Score.session_id)
Question --1:M--> Score (Score.question_id, optional for imported CSV rows)

Player(id, name) --1:M--> Score(id, session_id FK, player_id FK, category_id FK, question_id FK, is_correct, points) <--M:1-- GameSession(id, played_at)

3. Feature Checklist

Check off which mandatory requirement each planned feature will satisfy:

Planned Feature	Requirement(s) It Satisfies
Question hierarchy (abstract Question with Science, History, and Entertainment subclasses) and recursive category-tree walk	OOP and recursion
Game engine: Queue serves questions in order, Stack provides “undo last answer”	Queue and Stack
SQLite database layer: Player, GameSession, Question, Category, and Score tables with DAO classes using PreparedStatement	Database and ER model
Open Trivia Database API client with JSON parsed into Question objects	REST API
Data mining and charts: average score per category, most-missed questions, score trend, shown as bar and line charts	Data mining, visualization and tests on AnalyticsService)
JavaFX GUI (buttons, category ComboBox, player-name TextField, ListView/TableView) with CSV import and API fetch running on background threads and a progress bar	GUI with + wired controls , concurrency

4. Task Split

Member	Primary Responsibility (this milestone)
John Daevin S. Simbajon	Project and package structure, schema.sql, .gitignore, shared Eclipse build path (User Library), GitHub repo and branch rules, empty JavaFX MainApp shell, and reviewing pull requests
Gebrielle P. Mirasol	trivia.model package: abstract Question class, ScienceQuestion, HistoryQuestion, EntertainmentQuestion (each with its own scoring rule), plus Player and Category classes
Justine B. Cuizon	trivia.db: Database connection class (runs the schema, enables foreign keys) and the DAO classes (PlayerDao, QuestionDao, CategoryDao, ScoreDao) using PreparedStatement. trivia.api: Open Trivia Database client with HttpClient, JSON parsing, URL-decoding, and converting results into Question objects
Clarence Philip A. Elegino	trivia.game package: GameEngine (Queue serves questions in order, Stack for "undo last answer"), the recursive category-tree method, and JUnit tests for these classes
Joshua Malaya	trivia.analytics package: checks that the existing AnalyticsService and its 6 tests still work with the new schema, then drafts the mining methods (average score per category, most-missed questions, score trend) and the test plan

5. Public API You Plan to Use

(Name + one sentence on what data it provides)

Open Trivia Database (OpenTDB), https://opentdb.com/api.php
It is a free public REST API that returns multiple-choice and true/false trivia questions as JSON, with the category, difficulty, correct answer, and incorrect answers for each question. The app fetches these questions and parses them into Question objects (Science, History, or Entertainment) for the game.

6. Risks / Open Questions

(Anything you're unsure how to implement yet — flag it now so the instructor can help during Week 8-9 consultation time)

- Is the programming language for this project strictly java or can we use another language, like css, xml and others?